

A Extended Background and Mathematical Formulations

This section provides the detailed mathematical formulations and extended background for Battery-Free Sensor Nodes (BFNs), intermittent computing, and energy-aware primitives, which support the theoretical foundation of the EnTrust architecture discussed in Section II of the main manuscript.

A.1 Battery-Free Sensor Nodes and Intermittent Computing

Battery-Free Sensor Nodes (BFNs) operate by harvesting ambient energy from sources such as radio frequency (RF), solar radiation, vibration, or thermal gradients. The harvested power $P_h(t)$ is inherently time-varying and can be modeled as a stochastic process:

$$P_h(t) = \eta \cdot P_{src}(t), \quad (1)$$

where η denotes the energy conversion efficiency and $P_{src}(t)$ represents the ambient energy source intensity.

Energy is temporarily stored in capacitors or supercapacitors. The stored energy $E(t)$ in a capacitor C with voltage $V(t)$ is given by:

$$E(t) = \frac{1}{2}CV^2(t). \quad (2)$$

A node becomes operational only when $E(t)$ exceeds a minimum operational threshold E_{on} :

$$E(t) \geq E_{on}. \quad (3)$$

BFNs follow a **charge-store-compute** execution model. During the charging phase, energy accumulates until the voltage reaches a turn-on threshold V_{on} ; during computation, energy is consumed until it drops below a cutoff threshold V_{off} . The usable energy per cycle is thus:

$$E_{usable} = \frac{1}{2}C(V_{on}^2 - V_{off}^2). \quad (4)$$

Due to limited E_{usable} , tasks may not complete within a single cycle, leading to **intermittent computing**. If a task requires energy E_{task} :

$$E_{task} > E_{usable} \Rightarrow \text{task interruption occurs.} \quad (5)$$

To maintain forward progress, systems adopt checkpointing or idempotent execution models. The expected number of re-executions for a task can be approximated as:

$$N_{retry} \approx \left\lceil \frac{E_{task}}{E_{usable}} \right\rceil. \quad (6)$$

This intermittent behavior introduces challenges in preserving execution state, ensuring data consistency, and minimizing redundant computation.

A.2 Energy-Aware Task Scheduling

In BFNs, task scheduling must explicitly incorporate energy dynamics. Let $E_{res}(t)$ denote the residual energy and E_i the energy required for task i . A basic feasibility condition for task execution is:

$$E_{res}(t) \geq E_i + E_{safe}, \quad (7)$$

where E_{safe} is a safety margin to prevent brownout during execution.

The scheduler aims to maximize system utility under energy constraints. This can be formulated as an optimization problem:

$$\max \sum_i U_i x_i \quad (8)$$

subject to:

$$\sum_i E_i x_i \leq E_{budget}, \quad x_i \in \{0, 1\}, \quad (9)$$

where U_i represents the utility (e.g., priority or importance) of task i , and E_{budget} is the available energy within a scheduling window.

Dynamic Voltage and Frequency Scaling (DVFS) further enables fine-grained energy control. The dynamic power consumption is:

$$P_{dyn} = \alpha C_L V^2 f, \quad (10)$$

where α is the switching activity factor, C_L is the load capacitance, V is the supply voltage, and f is the clock frequency.

Execution time and energy consumption exhibit a trade-off:

$$E_{task} = P_{dyn} \cdot T_{exec}, \quad T_{exec} \propto \frac{1}{f}. \quad (11)$$

Thus, reducing V and f lowers instantaneous power but may increase total energy due to prolonged execution under leakage, necessitating careful scheduling decisions.

A.3 Energy-Aware Security Primitives

Energy constraints also significantly impact security design, requiring primitives that are both lightweight and interruption-resilient.

Physically Unclonable Functions (PUFs) generate device-specific responses based on manufacturing variations. A PUF can be modeled as:

$$R = f_{PUF}(C, \delta), \quad (12)$$

where C is the challenge input and δ captures physical randomness. Due to environmental noise, repeated evaluations may produce slightly different responses:

$$R' = R \oplus n, \quad (13)$$

where n denotes noise. Reliability is typically quantified using intra-Hamming distance:

$$HD_{intra} = \mathbb{E}[d_H(R, R')], \quad (14)$$

necessitating error correction mechanisms such as fuzzy extractors.

Atomic Lightweight Cryptographic Library decomposes cryptographic operations into fine-grained atomic units. Let a cryptographic task consist of k atomic operations:

$$\mathcal{C} = \{op_1, op_2, \dots, op_k\}. \quad (15)$$

Each atomic operation op_j has bounded energy consumption:

$$E(op_j) \leq E_{atomic}^{max}. \quad (16)$$

This ensures that:

$$E_{atomic}^{max} < E_{usable}, \quad (17)$$

allowing each operation to complete within a single energy cycle.

Security strength can be modeled as a function of executed rounds:

$$S = g(r), \quad (18)$$

where r is the number of completed rounds. Under energy constraints, the system dynamically selects r such that:

$$r = \max\{r' \mid E(r') \leq E_{res}(t)\}. \quad (19)$$

This leads to an adaptive security policy:

$$\text{maximize } S \quad \text{subject to } E_{crypto} \leq E_{res}(t), \quad (20)$$

balancing confidentiality and energy availability. Overall, the atomic and reconfigurable design enables BFNs to maintain secure operation under intermittent power, providing a practical trade-off between energy consumption, execution progress, and cryptographic strength.

B Ultra-Lightweight Atomic Encryption Generation

This section details the hardware and logical implementation of the adaptive encryption mechanism discussed in Section III of the main manuscript.

B.1 Sampling-Free Energy Level Monitor Circuit

BFNs generally face the problem of energy scarcity, as their power supply depends on intermittent and unstable ambient energy harvesting sources such as light, RF, or vibration. However, traditional energy sensing methods rely on ADCs and MCUs for voltage sampling and calculation. Even if the node

operates in a low-power sleep state, it still needs to periodically wake up for energy monitoring, resulting in high energy overhead. To combat this difficulty, we design a micro-power energy level monitor circuit (Fig 1) that enables real-time monitoring of the energy state of the node without requiring power-hungry clock-driven components like MCU or ADC. The circuit comprises two primary components, *i.e.*, Ultra-low-power Amplifier (AMP) and an Energy Level Encoding module, as follows.

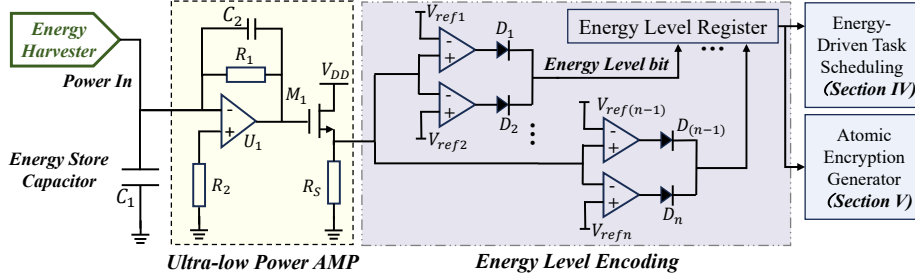


Figure 1: Sampling-free energy level monitor circuit.

B.1.1 Ultra-low-power Amplifier

Since the voltage signal from the storage capacitor is often small in magnitude, direct sampling can suffer from low accuracy. The Charge Amplifier (CPA) first amplifies the input signal, with its gain determined by the input capacitance C_{in} and feedback capacitance C_f , described by:

$$V_{out} = \frac{Q_{in}}{C_f} = \frac{C_{in} \cdot V_{in}}{C_f} \quad (21)$$

Here, V_{out} is the amplified output and Q_{in} is the input charge. Proper selection of C_f and C_{in} enables tunable gain with minimal power overhead.

To ensure stable signal delivery to the comparator, the CPA output is buffered using a low-leakage CMOS voltage follower, which provides high input impedance and low output impedance, approximated by $V_{out} \approx V_{in}$.

B.1.2 Computation-Free Energy State Encoding

To address the challenge of accurately perceiving the energy state of terminal nodes under severe environmental energy fluctuations, which often causes failures in task scheduling and security policy switching, this design introduces a window comparator array. The system configures multiple reference voltage thresholds $V_{ref1} \sim V_{ref4}$ on the integral output voltage $V_{out}(t)$ and continuously compares the real-time voltage levels. When $V_{out}(t)$ crosses a threshold, the corresponding comparator is triggered, resulting in a state transition at the output. This design enables high-resolution energy segmentation and accurate identification of the node's energy state, providing a reliable basis for subsequent scheduling and security decisions.

The proposed circuit precisely partitions the available energy range into five mutually exclusive levels and generates a unique encoding for each level, such as one-hot or binary representations. By leveraging this structured energy segmentation and flexible encoding control mechanism, the system is able to continuously monitor energy states and promptly react to energy variations, even when the MCU remains in a low-power sleep state.

With the proposed design, the system achieves real-time and high-precision sensing of environmental energy states under ultra-low power consumption. These energy states are rapidly converted into level codes recognizable by the control logic, enabling the dynamic activation of an ultra-lightweight, atomic cryptographic function library to generate optimal security decisions. Meanwhile, the adaptive task scheduling mechanism is also triggered, resulting in a robust, energy-aware system that effectively balances security with overall energy efficiency across diverse and dynamic energy-harvesting scenarios.

B.2 Energy-Aware Security Decision Mechanism (MDSM)

Table 1: Key indicators and their classifications

Indicator	Description	Value Range and Explanation
Energy State (E)	Current available energy level of the node	E_1 (Very Low) to E_5 (Sufficient)
Task Load (L)	Complexity of the current processing task	L_1 (Low), L_2 (Medium), L_3 (High)
Data Privacy (P)	Sensitivity of task data	P_1 (Low), P_2 (Medium), P_3 (High)
Threat Level (T)	Detected security threat level in environment	T_1 (Low), T_2 (Medium), T_3 (High)
Node Role (R)	Functional importance of the node within the network	Controller > Relay > Sensor

B.2.1 Input Indicator Normalization

The system selects five key input indicators, including energy state (E), task load (L), data privacy (P), threat level (T), and node role (R), which respectively characterize the node's residual energy, task complexity, data sensitivity, environmental threat intensity, and functional importance within the network. Since these indicators differ in physical meaning and scale, direct aggregation may lead to inconsistency. To ensure comparability and computational stability, all indicators are quantitatively modeled as continuous variables normalized to the range $[0, 1]$ based on measurable system parameters.

Specifically, all indicators are derived from observable system states, ensuring clarity and reproducibility. The energy state (E) is designed as a composite indicator that captures both residual energy and environmental energy availability through energy harvesting dynamics. Instead of explicitly modeling environmental factors such as solar intensity, RF energy density, or vibration frequency, these characteristics are implicitly reflected in the measured voltage level and charging behavior of the energy storage unit. Accordingly, the energy state is quantified as

$$E^* = \alpha \cdot \frac{V - V_{\min}}{V_{\max} - V_{\min}} + (1 - \alpha) \cdot \frac{\frac{\Delta V}{\Delta t}}{\left(\frac{\Delta V}{\Delta t}\right)_{\max}}, \quad (22)$$

where V denotes storage voltage and $\Delta V/\Delta t$ represents the charging rate from an ultra-low-power energy detection circuit. The parameter $\alpha \in [0, 1]$ balances residual energy and energy harvesting dynamics.

The task load (L) reflects both computational and communication demands, and is quantified using the estimated CPU cycles and data transmission volume:

$$L^* = \beta \cdot \frac{C}{C_{\max}} + (1 - \beta) \cdot \frac{D}{D_{\max}}, \quad (23)$$

where C and D denote the required computation and communication resources, respectively, and $\beta \in [0, 1]$ controls their relative importance.

The data privacy indicator (P) is defined based on the semantic sensitivity of the processed data and is mapped to a normalized value as $P^* = \frac{s(\text{type})-1}{2}$, where $s(\text{type})$ represents the sensitivity level associated with the data type.

The threat level (T) is quantified by aggregating multiple observable security-related metrics, including abnormal communication frequency, authentication failure rate, and interference intensity:

$$T^* = w_1 \cdot \frac{f_a}{f_{\max}} + w_2 \cdot \frac{r_f}{r_{\max}} + w_3 \cdot \frac{I}{I_{\max}}, \quad (24)$$

where w_i are weighting coefficients satisfying $\sum_i w_i = 1$.

Finally, the node role (R) reflects the functional importance of a node in the network topology and is normalized as $R^* = \frac{R-1}{2}$. Through the above formulation, all indicators are represented as normalized continuous variables within a unified range, effectively eliminating dimensional heterogeneity and enabling stable and consistent weighted aggregation in subsequent decision-making processes.

B.2.2 Entropy Weight Method for Weight Calculation

To ensure objective, data-driven indicator weighting, the Entropy Weight Method (EWM) is adopted over subjective alternatives like the Analytic Hierarchy Process (AHP). Unlike AHP, which relies on expert judgment and risks bias, EWM derives indicator importance from intrinsic data statistics. This suits battery-free nodes (BFNs), where states such as energy availability and threat level fluctuate dynamically and unpredictably. By quantifying indicator dispersion, EWM assigns higher weights to those with greater variability, reflecting their higher information content and decision contribution.

The entropy values reported in Table 2 are computed from a comprehensive dataset encompassing 10,000 operational cycles of the prototype system. All cycles were obtained from real measurements on the implemented prototype rather than from simulation or synthetic data. The dataset was collected over a 30-day period under diverse energy harvesting conditions, including varying intensities of solar radiation (100–1000 W/m²), RF power (-20 to 0 dBm), and vibration sources. Each cycle records a normalized indicator vector (E, L, P, T, R), capturing dynamic variations in energy states, task characteristics, and environmental

threats. The information entropy of each indicator is calculated from this historical matrix, where lower entropy indicates higher data dispersion. Based on these values, the information utility is derived and normalized to obtain the final weights.

Table 2: Entropy values and weights of indicators

Indicator	Entropy	Information Difference	Weight
E	0.8202	0.1798	0.3202
L	0.9128	0.0872	0.1554
P	0.8845	0.1155	0.2057
T	0.9059	0.0941	0.1676
R	0.9152	0.0848	0.1511

To verify the robustness of the derived weights, a sensitivity analysis was conducted. The dataset was partitioned into subsets based on specific energy sources (solar-only vs. RF-only) and varying sample sizes (2,000 to 10,000 cycles). As shown in Table 3, the resulting EWM weights remain highly stable, with a maximum fluctuation of less than 3.5% for the most critical indicator (E). This confirms that the calculated weights are statistically robust and not overly sensitive to transient environmental anomalies or dataset partitions, ensuring reliable long-term operation of BFNs.

Table 3: Sensitivity Analysis of EWM Weights under Different Dataset Conditions

Dataset Condition	E	L	P	T	R
Full Dataset (10,000 cycles)	0.3202	0.1554	0.2057	0.1676	0.1511
Solar-only Environment	0.3145	0.1582	0.2014	0.1710	0.1549
RF-only Environment	0.3312	0.1510	0.2105	0.1623	0.1450
50% Random Sample (5,000 cycles)	0.3215	0.1548	0.2061	0.1665	0.1511

To validate the effectiveness of the EWM-based allocation, it is compared against an Equal Weight (EQW) baseline, where all five indicators are assigned a fixed weight of 0.2. Table 4 shows that EWM significantly outperforms EQW. By dynamically prioritizing highly variable and critical states—such as the energy state (E , 0.3202) and priority (P , 0.2057)—the BFN system achieves more risk-aware decision-making. Compared to EQW, EWM reduces energy depletion events and improves the overall task success rate, demonstrating its practical advantage for energy-aware decision-making.

This methodology ensures that the assigned weights accurately reflect real-world system dynamics rather than static assumptions. Compared with alternative techniques such as *principal component analysis* (PCA), which primarily focuses on dimensionality reduction, EWM maintains the interpretability of each indicator while highlighting their relative importance, making it highly suitable for lightweight, embedded decision-making mechanisms in BFNs.

Table 4: Performance Comparison: EWM vs. Equal Weight (EQW) Baseline

Metric	EQW (Equal Weight)	EWM (Proposed)	Improvement
Task Success Rate (%) $\uparrow$	78.5%	89.3%	+13.8%
Energy Depletion Rate (%) $\downarrow$	14.7%	4.2%	-71.4%
Average System Utility $\uparrow$	65.4	82.7	+26.5%
Dead Time Ratio (%) $\downarrow$	11.2%	3.1%	-72.3%

Runtime implementation of entropy-weight updating: The entropy-weight calculation is not executed during every security decision at the battery-free node. Instead, historical indicator samples are periodically aggregated and processed at the gateway, where the entropy values and the corresponding weight vector are recomputed at a relatively low frequency. The gateway then quantizes the five weights into a fixed-point representation and transmits the updated vector to the node.

During online operation, the node only reads the latest five weights from a local register and performs fixed-point multiply-accumulate operations to calculate the composite security score. Therefore, logarithmic operations, probability normalization, and historical-matrix storage are not required at the resource-constrained terminal. A weight update is applied only when the difference between the new and current weight vectors exceeds a predefined threshold, thereby avoiding unnecessary communication and configuration operations.

B.2.3 Composite Scoring Function

The final security scoring function adopts a weighted summation form:

$$\text{Score} = w_E E_{\text{norm}} + w_L L_{\text{norm}} + w_P P_{\text{norm}} + w_T T_{\text{norm}} + w_R R_{\text{norm}} \quad (25)$$

Where w_E, w_L, w_P, w_T, w_R are the entropy weight method derived weights with $\sum_i w_i = 1$, which ensures the stability of the scoring results. These quantification rules ensure that all indicators are not only comparable in scale but also physically interpretable, improving the transparency and reproducibility of the proposed mechanism.

To improve practical system adaptability, the continuous score output is mapped to one of eight predefined security profiles to facilitate simple runtime control. Although the continuous score in $[0, 1]$ accurately reflects risk, real terminal security mechanisms such as encryption strength and authentication complexity are typically implemented discretely. Thus, equal-width interval partitioning maps the score to one of eight predefined security profiles. This simplification reduces hardware complexity and maintains smooth policy transitions, improving system responsiveness and stability. The discretization scheme also offers extensibility for flexible granularity and threshold adjustment according to application scenarios, balancing security and resource constraints dynamically. The profile identifiers S1–S8 are implementation labels rather than a strict ranking of cryptographic strength.

B.3 Library of Encryption Algorithms

B.3.1 Atomic Operators and Formal Modeling

To enable formal, flexible, and runtime-reconfigurable construction, algorithms are represented as a library of reusable atomic operators:

$$A = \{f_1, f_2, f_3, f_4, f_5\}.$$

These operators correspond to elementary cryptographic primitives. The proposed operator library is sufficient for the lightweight block ciphers considered in this work. A new cipher can be instantiated without additional hardware only when its nonlinear transformation, diffusion layer, key-schedule operations, and control flow can be completely expressed by the existing operator set. Otherwise, the library can be extended by introducing additional atomic operators.

- **Nonlinear Mapping** $f_1(\cdot)$: Implements configurable S-Box-based nonlinear substitution for SPN ciphers such as PRESENT and GIFT. Alternative S-Box implementations (e.g., 8-bit) can be incorporated when required by the target cipher.
- **Linear Diffusion** $f_2(\cdot)$: Implements linear diffusion operations such as bit permutation, shift, or rotation used by lightweight block ciphers.
- **Modular Addition** $f_3(x, y)$: Performs modular addition

$$f_3(x, y) = (x + y) \bmod 2^n.$$

- **Bitwise XOR** $f_4(x, y)$: Performs

$$f_4(x, y) = x \oplus y.$$

- **Cyclic Rotation** $f_5(x, r)$: Performs parameterized left or right cyclic rotation by r bits.

Algorithm execution is realized by chaining the required atomic operators according to the selected cipher specification. For illustration, an ARX-based round can be represented as

$$Y_k = f_5(f_4(f_3(X_{k-1}, K_k), K_k), r),$$

where X_{k-1} and Y_k denote the input and output round states, K_k is the round subkey, and r is the rotation distance. The above expression illustrates the composition of reusable operators rather than the exact round function of any specific cipher.

B.3.2 Hardware-Level Modular Implementation

At the hardware level, each atomic operator is mapped to a reusable IP core with a lightweight streaming interface, similar to AXI-Stream, where input data (`data_in`) and key (`key_in`) are provided alongside control signals such as `valid`, `ready`, and `start`, and the processed output is delivered through `data_out`. This standardized interface enables seamless pipelining, time-multiplexed execution, and cross-algorithm reuse. All modules are orchestrated by a centralized control unit, which selects among pre-instantiated operators through configuration registers, enable signals, and hardware multiplexers according to runtime energy availability and security requirements. Flash/ROM is used only to store constants, profile parameters, or lookup tables, rather than to load cryptographic hardware modules at runtime.

B.3.3 Dynamic Operator Composition and Scheduling

Combined with a micro-energy-aware security decision mechanism, the control logic dynamically selects an optimal encryption path based on node energy state, data size, and security level. Unlike static monolithic designs, this allows fine-grained trade-offs between encryption strength and energy consumption, maximizing node lifetime and maintaining multi-level security under ultra-low-power constraints.

B.3.4 Architectural Advantages

The proposed library follows a plug-and-play paradigm. Adding a new algorithm (e.g., SIMON or GIFT) does not require new hardware; only the FSM in the control unit needs updating to invoke existing operators in a new sequence. Resource sharing across f_2 – f_6 reduces silicon area and static power, while parameterized module design ensures adaptability to different algorithms.

Table 5: Typical module sequences of lightweight encryption algorithms

Algorithm	Representative Atomic Modules	Rounds
PRESENT	S-Box $\rightarrow$ P-Box $\rightarrow$ XOR Round Key	31
GIFT	S-Box $\rightarrow$ P-Box $\rightarrow$ XOR Round Key	28
LED	S-Box $\rightarrow$ Shift/Rotate $\rightarrow$ XOR Round Key	48
SPECK	ROR $\rightarrow$ ADD $\rightarrow$ XOR Round Key	32
SIMON	AND $\rightarrow$ ROL $\rightarrow$ XOR Round Key	44
T-HIGHT	ADD $\rightarrow$ XOR $\rightarrow$ ROL	32

Table 5 illustrates the flexibility and generality of the modular design. All module sequences are dynamically constructed from atomic operators, demonstrating scalability, hardware efficiency, and adaptability for energy-constrained, intermittent computing environments.

C Energy-Aware Task Scheduling and Energy Models

This section details the task models, comprehensive energy consumption models, and the low-level hardware implementation of the HEDA architecture, which serve as the basis for the energy-aware scheduling decisions in Section IV of the main manuscript.

C.1 Task Model

In this framework, application tasks are modeled as a directed acyclic graph (DAG) $G = (V, E)$, where V represents the set of subtasks and E denotes the data or control dependencies among them. For each subtask $v \in V$, the following attributes are defined:

- **Computation workload $c(v)$** : Represents the required CPU cycles for subtask v , reflecting its computational complexity.
- **Data size $d(v)$** : Denotes the amount of data to be processed or transmitted by the subtask.
- **Computational density $\rho(v)$** : Defined as $\rho(v) = \frac{c(v)}{d(v)}$, it characterizes the computation-to-communication ratio of a subtask. Higher values favor local execution, whereas lower values favor gateway offloading.
- **Deadline $t(v)$ and slack time $s(v)$** : The deadline $t(v)$ represents the latest allowable finish time of the subtask. The slack time, defined as $s(v) = t(v) - S(v) - T(v)$, represents the maximum tolerable delay for scheduling purposes. Here, $S(v)$ denotes the earliest start time of subtask v , determined by the completion of all its predecessor subtasks, and $T(v)$ represents the estimated execution and data transmission time of v . Slack time guides dynamic scheduling and priority assignment.
- **Priority $p(v)$** : Reflects the importance or urgency of the subtask, allowing trade-offs between real-time performance and energy efficiency.
- **Memory and I/O requirements $m(v)$** : Specifies the memory and I/O resources required by the subtask, which are critical for scheduling under resource-constrained conditions.

Transmission delay is explicitly considered in this model, including bandwidth-dependent transfer time and protocol overhead, which are incorporated into $T(v)$.

C.2 Comprehensive Energy Consumption Models

In energy-harvesting or battery-free terminal devices, the available energy supply is not only highly constrained but also exhibits strong temporal intermittency. The total energy dissipation of a battery-free node can be classified

into two major functional domains: communication-related energy E_{comm} and on-device computation energy E_{local} .

$$E_{\text{total}} = E_{\text{comm}} + E_{\text{local}}. \quad (26)$$

C.2.1 Communication Energy Consumption Modeling

The total communication energy is decomposed according to:

$$E_{\text{comm}} = E_{\text{tx}} + E_{\text{rx}} + E_{\text{ctrl}} + E_{\text{sec}},$$

where each term captures a distinct category of communication operations.

Transmission Energy Consumption: The energy for transmitting N packets of payload size L is given by

$$E_{\text{tx}} = N \times \left((P_c + \eta \cdot P_{\text{te}}) \times \frac{L}{R} \right), \quad (27)$$

where the transmission duration $\frac{L}{R}$ encapsulates the impact of physical-layer rate R on temporal energy cost. The circuit power P_c contains a static baseline component P_{base} and a modulation-related dynamic component P_{mod} :

$$P_c = P_{\text{base}} + P_{\text{mod}}, \quad P_{\text{mod}} \propto R^\beta, \quad \beta \approx 1. \quad (28)$$

To maintain a target SNR at the receiver, the required transmission power must compensate for propagation loss:

$$P_{\text{te}} = \frac{\text{SNR} \cdot N_0 \cdot d^\alpha}{\eta \cdot h}. \quad (29)$$

Power amplifier efficiency follows a saturation response:

$$\eta = \eta_{\text{max}} \cdot (1 - e^{-\gamma P_{\text{te}}}). \quad (30)$$

The payload size and data rate are computed as:

$$L = \left(\left\lceil \frac{\text{size}}{\text{MTU}} \right\rceil \cdot B + \text{size} \right) \cdot 8, \quad (31)$$

$$R = B \cdot \log_2 \left(1 + \frac{S}{N + N_{\text{interf}}} \right) \cdot C. \quad (32)$$

Reception Energy Consumption: The reception energy model is

$$E_{\text{rx}} = N \cdot \left(P_{\text{rx}} \cdot \frac{L}{R} \right) + P_{\text{idle}} \cdot T_{\text{idle}}. \quad (33)$$

Low-Power Listening (LPL) replaces continuous reception with periodic channel sampling:

$$E_{\text{rx}}^{\text{LPL}} = N \cdot \left(P_{\text{rx}} \cdot \frac{L}{R} \right) + (P_{\text{wake}} T_{\text{wake}} + P_{\text{sleep}} T_{\text{sleep}}) \frac{T_{\text{idle}}}{T_{\text{cycle}}}. \quad (34)$$

Protocol Control Overhead: Control signaling includes handshake packets, channel probing, ACK/NACK exchange, and synchronization preambles:

$$E_{\text{ctrl}} = N_{\text{ctrl}} \left(\frac{P_{\text{tx}}C}{R} + \frac{P_{\text{rx}}C}{R} \right) + E_{\text{proc}}N. \quad (35)$$

In LPL-enabled nodes, additional wake-ups multiply this overhead:

$$E_{\text{ctrl}}^{\text{LPL}} = N_{\text{wake}} \left[N_{\text{ctrl}}^{\text{per}} \frac{(P_{\text{tx}} + P_{\text{rx}})C}{R} + E_{\text{proc}}N_{\text{ctrl}}^{\text{per}} \right]. \quad (36)$$

Secure Transmission Overhead: Security operations introduce substantial computational energy:

$$E_{\text{sec}} = N \cdot (E_{\text{enc}} \cdot L + E_{\text{dec}} \cdot L + E_{\text{auth}}). \quad (37)$$

If the system supports configurable security levels l , the generalized model becomes

$$E_{\text{sec}} = N \cdot (E_{\text{enc}}(l)L + E_{\text{dec}}(l)L + E_{\text{auth}}(l)). \quad (38)$$

C.2.2 Modeling of Local Processing Energy Consumption

Local energy consumption originates from processor switching activities, memory accesses, leakage currents, and operation of peripheral units.

System Load Power Consumption: The processor's power consumption consists of three major components:

$$P_{\text{dyn,sys}}(f) = \alpha C_{\text{eq}} V_{\text{op}}^2 f, \quad (39)$$

$$P_{\text{ind}} = P_{\text{mem}} + P_{\text{IO}} + P_{\text{aux}}, \quad (40)$$

$$P_{\text{sta}} = V \cdot I_{\text{leak}}. \quad (41)$$

Task Execution Time: For parallelizable operations:

$$T = \max \left(\frac{C}{f_{\text{cpu}}}, \frac{D_{\text{tx}} + D_{\text{rx}}}{B}, \frac{S}{R_{\text{sto}}} \right), \quad (42)$$

while serial operations satisfy:

$$T = \frac{C}{f_{\text{cpu}}} + \frac{D_{\text{tx}} + D_{\text{rx}}}{B} + \frac{S}{R_{\text{sto}}}. \quad (43)$$

C.3 Low-Level Hardware Implementation of the HEDA Architecture

This section provides the detailed hardware implementation of the Hardware-Embedded Dynamic Adjustment (HEDA) architecture mentioned in Section IV.B of the main manuscript. It details the internal logic of the DVFS Manager, Voltage Controller, and Frequency Controller.

C.3.1 DVFS Scheduling Controller Overview

For BFNs, DVFS-enabled components—such as processing cores and task schedulers—generate voltage–frequency requests based on real-time energy states and workload demands. The DVFS controller comprises a manager, a voltage controller, and a frequency controller, which jointly orchestrate voltage–frequency adjustments to adapt system operation to dynamically varying energy conditions.

Upon receiving a request, the Manager consults a software-configured VF lookup table to obtain the target voltage and frequency for the desired performance level. It compares the current and target voltages to decide whether scaling is required, then sends commands to the voltage and frequency controllers.

The Voltage Controller adjusts supply levels by programming an off-chip PMIC via I²C, while the Frequency Controller configures PLL clock sources to reach the target operating frequency. Together, these controls enable BFNs to autonomously adapt system operation under limited energy availability and dynamically varying workloads.

C.3.2 DVFS Manager — Unified Request Management Unit

In BFNs, the system core module (DVFS MCU) generates voltage–frequency scaling requests based on energy states and computational loads, specifying a target performance level. The DVFS Manager then consults a software-configured voltage–frequency lookup table (VF LUT) to retrieve the corresponding parameters. It compares these with the current system state to determine if scaling is required, and if so, issues commands with target parameters to the voltage and frequency Controllers.

To ensure stable operation, a configurable delay mechanism enforces that frequency switching occurs only after the voltage adjustment has fully settled, thereby mitigating potential operational risks. The DVFS Manager integrates several key functions:

- **Performance Level Decoding:** Receives DVFS requests and target levels, looks up voltage and frequency from a table, and compares with the current voltage to decide increase, decrease, or hold.
- **Control State Machine:** Applies priority logic (voltage up before frequency up, frequency down before voltage down) for stable transitions, with delay states to await voltage or frequency stabilization after writing to the PMIC register.
- **Timing Control:** Uses the `delay_cycles` register and counters to manage delay intervals, where actual delay = `delay_cycles` × clock period T . State transitions are gated by completion signals for safe operation.
- **Fault Handling:** Detects anomalies such as voltage adjustment failures or operational timeouts. In the case of voltage mismatches, the module

sets `voltage_fault` and logs diagnostic information; for timeout events, a `timeout_cycles` counter is employed, and the system reverts to the idle state if the threshold is exceeded.

- **Asynchronous Interaction:** Ensures reliable communication across multiple clock domains by employing double flip-flop synchronizers. It manages critical signals, including `voltage_ack`, `voltage_fault`, and `freq_ack`, thereby preventing timing violations, metastability issues, and signal glitches.

C.3.3 Voltage Controller — Voltage Regulation Module

The voltage controller performs discrete voltage adjustments for BFNs. It receives requests and target values from the DVFS controller, programs the off-chip PMIC via I²C, and is managed by a state machine (Figure 2, Table 6). To ensure reliability, it supports masked writes and read-back verification, providing accurate transmission and error feedback to upper layers.

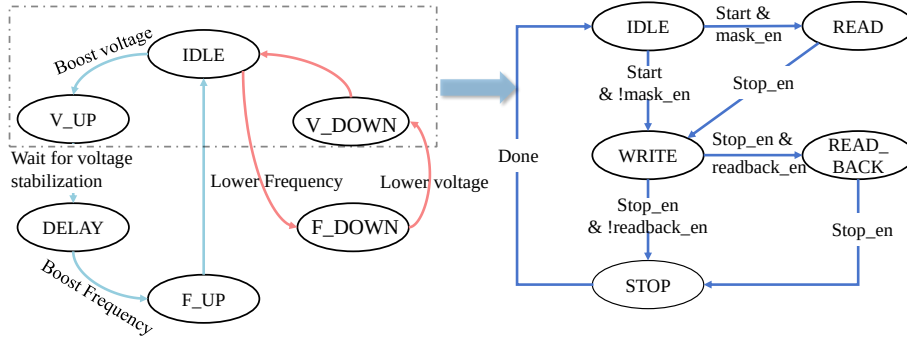


Figure 2: State machine of the voltage controller module, illustrating discrete voltage adjustment operations, communication with off-chip PMIC registers via I²C, and mechanisms for masked write and read-back verification.

An APB interface is provided, allowing the CPU to write start, stop, and acknowledge commands for I²C timing control into internal registers; address and data registers are used for PMIC register mapping and voltage data transmission.

Masked Write Mechanism: To prevent unintended modifications to unrelated fields in multi-bit PMIC registers, the module supports an optional masked write mechanism. This technique first reads the current register value (*mdata*) and then computes the actual value to be written (*wdata*) using the following equation:

$$wdata = (mdata \wedge mask_value) \vee (data0 \wedge \neg mask_value) \quad (44)$$

Finally, *wdata* is written back to the register. This method improves protocol compatibility and enhances portability.

Table 6: Descriptions of DVFS Control States

State Name	Function Description
IDLE	Idle state, waiting for a new DVFS adjustment request
VOLT_UP	Voltage increase phase, sending target voltage to the regulator module
DELAY_UP	Delay after voltage increase, waiting for voltage stabilization
FREQ_UP	Frequency increase phase, sending target frequency to the frequency module
FREQ_DOWN	Frequency decrease phase, reducing frequency as a priority
DELAY_DOWN	Delay after frequency decrease, waiting for frequency stabilization
VOLT_DOWN	Voltage decrease phase, lowering voltage accordingly

Read-back Verification: After writing the target voltage value (*data0*) to the PMIC register, the module reads back the register content (*rdata*) and compares it with *data0*. A match confirms that the write operation has completed correctly, whereas a mismatch asserts an error signal (*err* = 1) to report a write failure.

Status Feedback: The error signal *err* is synchronized with corresponding acknowledge signals and determined by active functional configuration. The specific conditions under which the error signal is generated are summarized in Table 7.

Table 7: Voltage Controller Error Output Conditions

Functionality	Error Condition	Error Output
Basic write only	I ² C bus no response or write failure	1
Read-back verification enabled	$rdata \neq data0$	1
Masked write enabled	I ² C write phase failure	1
Masked write and Read-back verification	$rdata \neq wdata$	1
Successful operation	Bus responds correctly, data matches	0

C.3.4 Frequency Controller — Clock Management Unit

The Frequency Controller enables dynamic performance adjustment for BFNs following energy restoration by supplying configurable clock signals to the DVFS MCU. It supports flexible clock source selection and single-stage frequency divi-

sion, ensuring low-power, glitch-free transitions while maintaining overall system stability.

Upon receiving frequency configuration commands (`freq_cfg`) from the DVFS manager, the frequency controller selects one of five external PLLs via glitch-free multiplexing, disabling the current clock before enabling the target. A lightweight divider, controlled by `freq_cfg[15:8]`, supports 1/2, 1/4, and 1/8 ratios using three cascaded T flip-flops. All outputs pass through the glitch-free selector, ensuring stable and reliable clock management for BFNs.

D Energy-Aware Authentication Framework and Protocols

To address the heterogeneity of IoT devices in terms of energy availability, computational capability, and security requirements [?, ?], this paper proposes an energy-aware hierarchical authentication framework.

D.1 Framework Definition and Design Principles

The framework provides a unified abstraction that organizes heterogeneous authentication primitives, including lightweight hashing, multifactor authentication, and PUF-based mechanisms, according to device energy states and hardware capabilities. This abstraction enables flexible and scalable identity authentication for devices ranging from passive sensors to moderately resource-constrained nodes under ultra-low-power conditions.

Based on this framework, three representative authentication protocols are developed for different operating regimes:

- **Lightweight Multi-factor Authentication Protocol (LMAP):** Designed for ultra-low-power and intermittently active nodes, this protocol leverages symmetric key primitives to reduce computational and communication overhead.
- **Light PUF-based Authentication Protocol (LPUFA):** This protocol employs PUF-based challenge-response mechanisms with lightweight error correction, making it suitable for hardware-constrained edge devices.
- **Enhanced Energy-aware Hash Authentication Protocol (E²HA):** Based on dynamic hash chains, this protocol adjusts authentication strength according to real-time energy availability.

The proposed framework is characterized by three key aspects: energy-awareness, capability matching, and security scalability. Authentication complexity is adapted to instantaneous energy availability, protocol selection is aligned with device hardware features (e.g., PUF support and storage capacity), and authentication strength is scaled according to security requirements.

Formally, the framework can be expressed as a mapping:

$$\mathcal{P} = f(E, C, S) \quad (45)$$

where E denotes the energy state, C represents device capability, and S indicates the required security level.

D.2 Lightweight Multi-factor Authentication Protocol (LMAP)

The lightweight multi-factor authentication protocol (LMAP) is designed for energy-constrained and intermittently powered sensor nodes, particularly battery-free nodes (BFNs) relying on ambient energy harvesting. To minimize communication and computational overhead, the protocol adopts a single-challenge single-response mechanism, while integrating cryptographic verification with physical-layer feature binding to achieve robust and low-cost authentication.

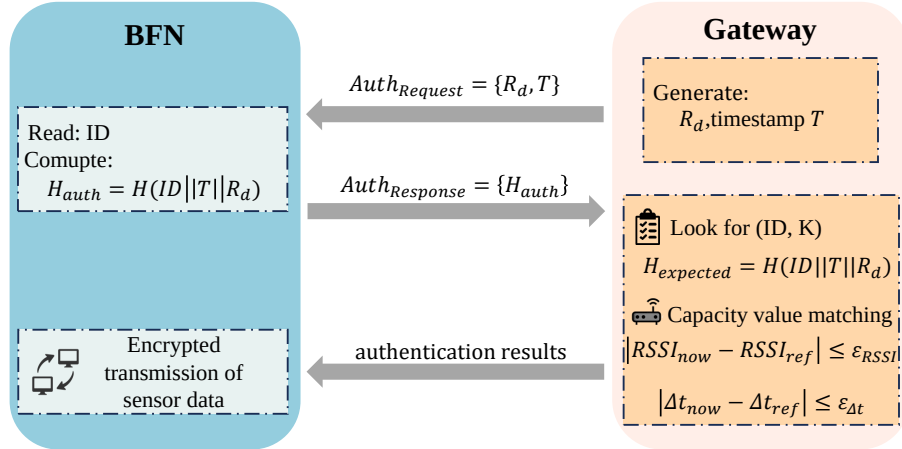


Figure 3: Overview of the LMAP authentication protocol for BFNs, showing the single broadcast-response interaction and lightweight security mechanisms.

D.2.1 Gateway Broadcasts Authentication Request

Due to the intermittent operation of BFNs caused by unstable energy availability (e.g., RF, light, or thermal sources), frequent communication should be avoided. Therefore, the gateway periodically or on-demand broadcasts a lightweight authentication request to trigger node responses without requiring continuous listening. The request is defined as

$$\text{Auth_Request} = \{R_d, T\}, \quad (46)$$

where R_d is a session-specific random challenge generated by the gateway, and T is a timestamp indicating the freshness of the request. The timestamp is validated within a predefined time window to prevent replay attacks.

D.2.2 Terminal Computes Response and Sends Upstream

Upon receiving the authentication request, the terminal node immediately computes a lightweight authentication response using its locally stored identifier ID and secret key K . The authentication hash is generated as

$$H_{\text{auth}} = H(ID \parallel K \parallel T \parallel R_d), \quad (47)$$

where $H(\cdot)$ denotes a hardware-friendly lightweight hash function. The terminal then transmits the response frame

$$\text{Auth_Response} = \{ID, H_{\text{auth}}\} \quad (48)$$

to the gateway. This design ensures that the gateway can directly identify the responding node without exhaustive search, thereby reducing verification latency and computational cost.

D.2.3 Gateway Performs Multi-factor Authentication Check

Upon receiving the response, the gateway first checks whether the timestamp T is within a valid time window to ensure message freshness. It then retrieves the corresponding credential pair (ID, K) from the whitelist and recomputes the expected hash value as

$$H_{\text{expected}} = H(ID \parallel K \parallel T \parallel R_d). \quad (49)$$

Authentication proceeds only if $H_{\text{expected}} = H_{\text{auth}}$, ensuring cryptographic correctness.

To further enhance security, physical-layer features are incorporated as an additional authentication factor. Specifically, the gateway measures the received signal strength (RSSI) and response delay Δt_{now} , and compares them with the reference values RSSI_{ref} and Δt_{ref} , which are established through historical averaging over multiple successful authentications. The verification conditions are given by

$$\begin{aligned} |\text{RSSI}_{\text{now}} - \text{RSSI}_{\text{ref}}| &\leq \varepsilon_{\text{RSSI}} \\ |\Delta t_{\text{now}} - \Delta t_{\text{ref}}| &\leq \varepsilon_{\Delta t} \end{aligned} \quad (50)$$

where $\varepsilon_{\text{RSSI}}$ and $\varepsilon_{\Delta t}$ are tolerance thresholds determined based on statistical variations under normal operating conditions.

Authentication is considered successful only when both cryptographic verification and physical-layer constraints are simultaneously satisfied. Otherwise, the request is rejected, and the event can be logged for anomaly detection or subsequent re-authentication.

D.2.4 Security Analysis and Correctness

The correctness of LMAP follows from the deterministic consistency of the hash function, such that

$$H(ID \parallel K \parallel T \parallel R_d)_{\text{device}} = H(ID \parallel K \parallel T \parallel R_d)_{\text{gateway}}. \quad (51)$$

Security is ensured via joint cryptographic and physical design. Timestamp T guarantees replay resistance by enforcing freshness constraints. Secret key K ensures adversaries cannot forge valid authentication responses without legitimate credentials, providing strong impersonation resistance. Integrating physical-layer features like RSSI and response delay binds logical identities to physical transmission characteristics, offering extra protection against device cloning and relay attacks. Consequently, LMAP achieves dual-factor authentication that balances security robustness and energy efficiency, suiting resource-constrained, intermittently powered IoT environments.

D.3 Lightweight Physical Unclonable Function Authentication Protocol (LPUFA)

LPUFA exploits the inherent uniqueness and low-power characteristics of PUF [?] circuits to realize a physically attack-resistant authentication scheme suitable for moderately energy-constrained devices. The protocol operates in two phases, namely registration and authentication, during which end devices are not required to store cryptographic keys or perform conventional encryption and decryption operations. Instead, identity verification is accomplished by leveraging pre-collected challenge-response pairs (CRPs) in conjunction with lightweight error-correction mechanisms. This design reduces energy consumption and computational overhead while preserving strong hardware adaptability and security assurances. The complete protocol flow is illustrated in Figure 4.

D.3.1 Offline Registration Phase

Before deployment, each terminal device must complete PUF modeling and identity binding within a secure offline environment. This phase is carried out collaboratively by the Verifier and the new Token, as described below:

- a) **ID Assignment and Initialization:** The Verifier generates a unique ID for each Token and writes it into the Token’s read-only memory.
- b) **CRP Collection:** The Verifier generates a pseudo-random set of challenges $\{C_1, \dots, C_m\}$ and applies them to the PUF circuit in the Token. Considering the potential impact of environmental factors such as voltage fluctuations and temperature variations, a *Jitter Measurement* mechanism is introduced to accurately characterize the stability of the PUF responses:
 - For each challenge C_i , the Verifier powers or resets the Token multiple times and collects the response $R_i^{(j)}$ for $j = 1, 2, \dots, K$.
 - Each bit’s stability is computed, and a reference response R_i^{ref} is formed via majority voting.
 - Unstable bits are explicitly labeled as low-confidence indicators to facilitate efficient error correction during authentication procedures.

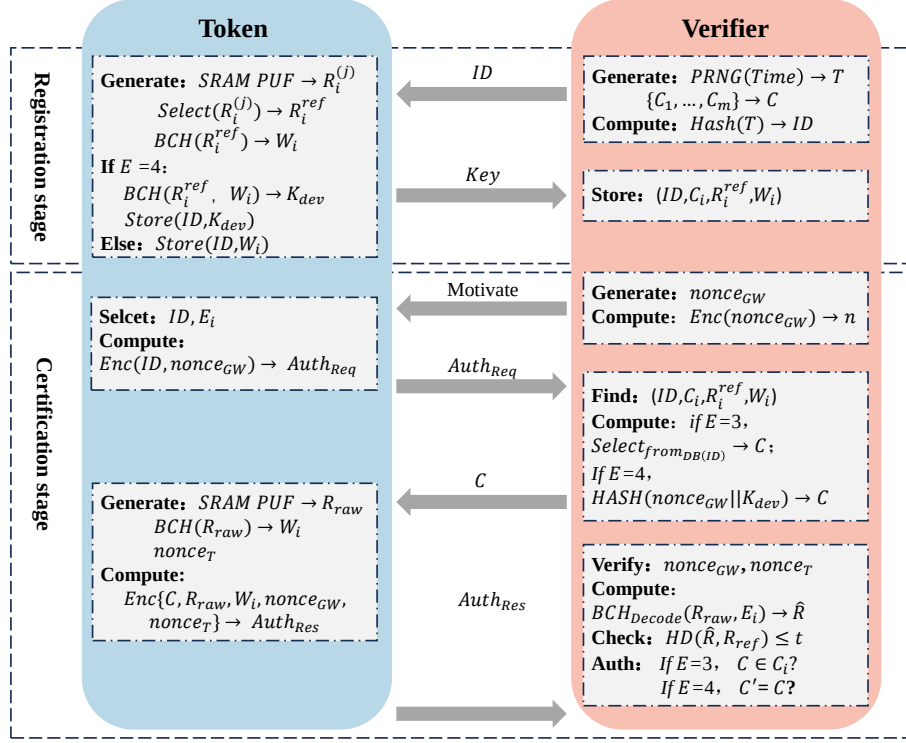


Figure 4: Protocol flow of the Lightweight PUF Authentication(LPUFA).

- c) **Helper Code Generation:** Considering bit errors in practical deployments, each R_i^{ref} is encoded using BCH codes to generate helper data W_i , with decoding threshold t negotiated during system initialization.
- d) **Database Storage:** The Verifier securely stores the tuple $\langle ID, C_i, R_i^{ref}, W_i \rangle$ in the database. The Token does not store any CRPs or helper data locally, thereby minimizing storage overhead and preventing physical extraction attacks.

After registration, the Token is deployed in the field. It does not store any cryptographic keys or CRPs, retaining only its device identifier and minimal lightweight hardware state. When external energy becomes available (e.g., via RF power), the Token is briefly powered on to compute real-time PUF responses, thereby minimizing both energy consumption and information leakage and making it particularly suitable for passive sensor node applications.

D.3.2 Interactive Authentication Phase

When a passive Token accesses the network or initiates task execution, it performs secure authentication under temporary external power, following a standard Challenge-Response protocol outlined below:

Table 8: Notation and Definitions in LPUFA Protocol

Symbol	Definition
Token	Passive device with embedded PUF; no long-term energy storage; operates only when externally powered.
Verifier	Authentication server with sufficient computation and storage resources.
ID	Unique identifier assigned to each Token by the Verifier.
C_i	The i -th challenge vector, randomly generated to stimulate the PUF.
$R_i^{(j)}$	Raw PUF response to C_i at the j -th sampling.
R_i^{ref}	Reference response for C_i , obtained via majority voting over multiple samplings.
C, R	Challenge and corresponding response of the PUF.
DB	Challenge-Response Pair (CRP) database storing $\langle ID, \{C_i, R_i\}_i \rangle$.
t	Error tolerance threshold to handle occasional jitter in PUF responses.
n	Number of challenges used per authentication session.

- a) **Auth Request:** On power-up, the Token reads its device identifier from non-volatile memory, generates a random session nonce nonce_T , and transmits the request:

$$\text{Auth_Request} = \{\text{ID}, \text{nonce}_T\} \quad (52)$$

- b) **Challenge:** Upon receiving the request, the Verifier retrieves the ID's record from the database, generates its own session nonce nonce_{GW} , and selects a challenge C_i along with its helper data W_i .

- *Level 3 (Standard):* Randomly select an unused tuple $\langle C_i, W_i \rangle$ from stored CRPs.
- *Level 4 (Dynamic):* Dynamically generate a new challenge C_i and predict R_i^{ref} using a pre-trained Strong PUF machine learning model at the Verifier side.

The Verifier then sends the challenge message:

$$\text{Auth_Challenge} = \{C_i, W_i, \text{nonce}_{GW}\} \quad (53)$$

- c) **Response:** The Token applies the received challenge C_i to its PUF circuit to obtain the raw response R_{raw} . It then uses the received helper data W_i

to correct noise-induced errors via lightweight BCH decoding, recovering the ideal response $R' = \text{BCH_Decode}(R_{raw}, W_i)$. To prevent exposing the PUF response in plaintext, the Token computes a cryptographic hash and sends the response:

$$V_T = \text{HASH}(R' \parallel \text{nonce}_{GW} \parallel \text{nonce}_T) \quad (54)$$

$$\text{Auth_Response} = \{V_T\} \quad (55)$$

- d) **Verification:** Upon receiving the response, the Verifier computes the expected hash value using its stored reference response R_i^{ref} :

$$V_V = \text{HASH}(R_i^{ref} \parallel \text{nonce}_{GW} \parallel \text{nonce}_T) \quad (56)$$

The Verifier checks if $V_T \stackrel{?}{=} V_V$. If they match, the Token is authenticated successfully.

This design decouples CRP collection from the runtime authentication process and adheres to the “compute offloading and storage delegation” philosophy. It provides robust security without storing cryptographic keys by leveraging uncloneable physical characteristics to resist cloning, replay, and modeling attacks, making it particularly suitable for low-power, passive IoT deployments.

D.3.3 Authentication Correctness

Authentication is accepted if and only if the following condition holds: $V_T = V_V$. This single verification step elegantly ensures that:

- **Authenticity:** The response originates from the genuine PUF instance capable of reproducing R_{raw} close enough to R_i^{ref} .
- **Robustness:** Noise-induced errors in R_{raw} are tolerable and successfully corrected by W_i within the BCH threshold t .
- **Freshness:** The inclusion of both nonce_T and nonce_{GW} in the hash prevents replay attacks and challenge forgery from malicious adversaries.

D.4 Enhanced Energy-Aware Hash Authentication Protocol (E²HA)

With the rapid proliferation of IoT devices, BFNs face stringent energy constraints, particularly in passive or semi-passive scenarios with unstable power and limited computing capability. Conventional authentication protocols impose heavy computation and long communication, leading to energy waste and frequent failures under fluctuating energy supply.

To address this, we propose E²HA, a lightweight HMAC-based authentication [?] protocol employing symmetric pre-shared keys for efficient identity binding and mutual authentication. The terminal energy level is dynamically

incorporated to adapt authentication behavior and enforce security policies, enabling full authentication only when sufficient energy is available. The complete protocol flow is shown in Figure 5, making E²HA well suited for low-power IoT deployments(e.g., wireless sensor networks and edge computing).

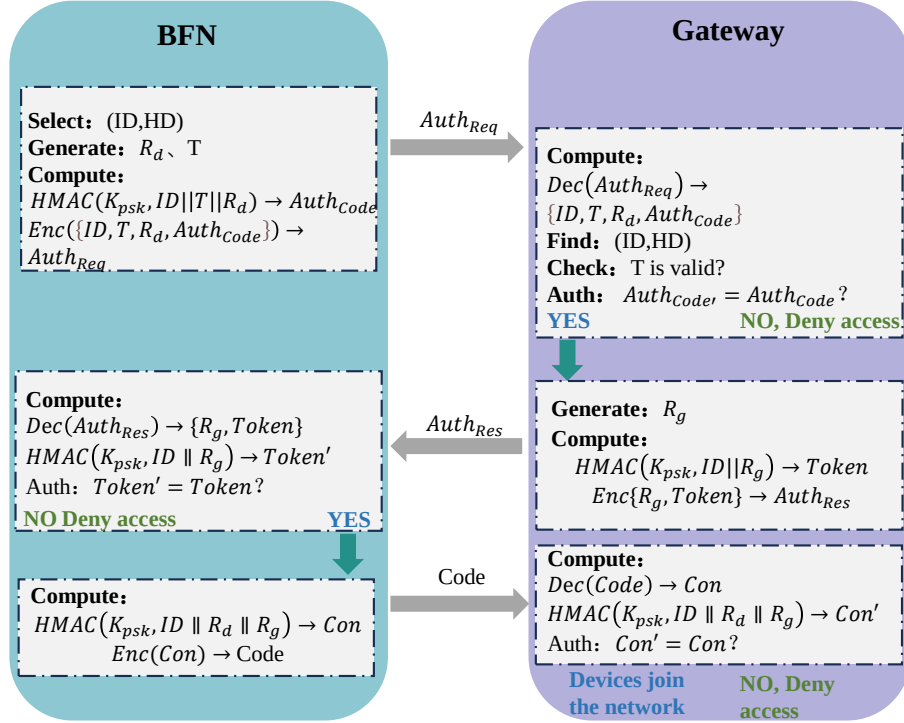


Figure 5: E²HA protocol flow, showing energy-aware mutual authentication between terminals and the gateway, with dynamic adaptation to terminal energy levels to reduce computation and communication overhead.

D.4.1 Initialization

The terminal device (Device) and the gateway (Gateway) share a unique device identifier ID and an HMAC pre-shared key K_{psk} (a fixed secret key) during the initialization phase.

D.4.2 Energy-Aware Authentication Policy

To explicitly incorporate energy awareness into the authentication process, the device dynamically selects its authentication mode based on its current energy level E .

$$\text{Mode} = \begin{cases} \text{Full Authentication,} & E \geq E_{\text{high}} \\ \text{Lightweight Authentication,} & E_{\text{low}} \leq E < E_{\text{high}} \\ \text{Deferred Authentication,} & E < E_{\text{low}} \end{cases} \quad (57)$$

Full authentication executes the complete three-way HMAC-based handshake, lightweight authentication reduces communication overhead by omitting the final confirmation from the device, and deferred authentication postpones the process until sufficient energy is available. This design aligns protocol execution with instantaneous energy availability, minimizing unnecessary failures and energy depletion.

D.4.3 Authentication Process

If $E < E_{\text{low}}$, the device enters the Deferred mode and remains in sleep state. Otherwise, the protocol proceeds as follows:

- a) **Device Sends Authentication Request:** The device generates a random nonce R_d , reads the current timestamp T , and transmits the request in plaintext to allow gateway lookup, accompanied by an HMAC for integrity and authenticity:

$$\text{Auth_Request} = \{ID, T, R_d, \text{Auth_Code}, \text{Mode}\} \quad (58)$$

where $\text{Auth_Code} = \text{HMAC}(K_{\text{psk}}, ID \parallel T \parallel R_d \parallel \text{Mode})$.

- b) **Gateway Verifies Device Identity:** Upon receiving **Auth_Request**, the gateway checks if the timestamp T is within a valid time window to prevent replay attacks. It retrieves K_{psk} using the ID and computes:

$$\text{Auth_Code}' = \text{HMAC}(K_{\text{psk}}, ID \parallel T \parallel R_d \parallel \text{Mode}) \quad (59)$$

If $\text{Auth_Code}' \neq \text{Auth_Code}$, the connection is rejected.

- c) **Gateway Generates Challenge Response:** To prove its authenticity to the device, the gateway generates a random nonce R_g and computes the HMAC token, binding it to the device's nonce R_d to ensure session freshness:

$$\text{Token} = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{"S2C"}) \quad (60)$$

Here, the constant string "S2C" (Server-to-Client) acts as a directional tag for domain separation. It ensures that the gateway's token cannot be maliciously reflected back by an attacker as a valid device confirmation code. The gateway then sends $\text{Auth_Response} = \{R_g, \text{Token}\}$.

- d) **Device Computes Token and Responds (Mode Dependent):** Upon receiving R_g , the device computes Token' locally. If it matches Token , the gateway is authenticated.

- If Mode == Lightweight, the device stops here to save transmission energy. Mutual authentication is implicitly achieved.
- If Mode == Full, the device replies with a confirmation message to explicitly finalize the handshake:

$$\text{Auth_Confirm} = \{\text{Confirm_Code}\} \quad (61)$$

where $\text{Confirm_Code} = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{"C2S"})$. The "C2S" (Client-to-Server) tag guarantees cryptographic separation from the gateway's token.

- e) **Gateway Verifies and Completes Authentication:** If the session is in Full mode, the gateway waits for **Auth_Confirm** and verifies:

$$\text{Confirm_Code}' = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{"C2S"}) \quad (62)$$

If the codes match (or if the session was successfully negotiated as Lightweight), the device is granted network access.

D.4.4 Energy-Aware Verification Logic

Authentication success in E²HA is jointly determined by cryptographic validity and the declared energy mode, forming a dual-domain verification mechanism. Formally, the gateway accepts the authentication if and only if:

$$\begin{cases} \text{Auth_Code}' = \text{Auth_Code}, \\ \text{Confirm_Code}' = \text{Confirm_Code} & \text{if Mode} = \text{Full}, \\ E \geq E_{\text{low}}, \end{cases} \quad (63)$$

and the device accepts the authentication if and only if:

$$\text{Token}' = \text{Token}. \quad (64)$$

This conditional verification ensures both security and energy-aware feasibility. Security correctness is guaranteed by the use of timestamps T and device nonces R_d to prevent replay attacks, while directional tags ("S2C" and "C2S") prevent reflection attacks by ensuring that messages generated by the gateway and the device produce distinct HMAC values even with identical input parameters. Execution feasibility is achieved by dynamically pruning communication overhead: when $E_{\text{low}} \leq E < E_{\text{high}}$, the device omits transmission of **Confirm_Code**, reducing energy consumption while maintaining baseline mutual trust. Together, these mechanisms enable E²HA to adapt securely to real-time energy availability without compromising authentication integrity.

D.5 Theoretical Security Analysis of Authentication Protocols

Based on the reviewers' suggestions, we provide a comprehensive theoretical security comparison of the three instantiated lightweight authentication protocols

(LMAP, LPUFA, and E2HA) discussed in Section V of the main manuscript. This analysis evaluates their resistance to common attack vectors in energy-constrained environments.

Table 9: Security capability comparison of lightweight authentication protocols

Attack Type	LMAP	LPUFA	E ² HA
Replay Attack	✓	✓	✓
Man-in-the-Middle	×	✓	✓
Impersonation Attack	✓	✓	✓
Key Leakage Attack	✓	✓	×
Replay and Delay Attack	✓	✓	✓
Physical Attack	×	✓	✓
Energy-Constrained Attack	✓	✓	✓
Redundant Authentication	✓	✓	✓
Eavesdropping	×	✓	✓
Tampering Attack	×	×	✓
Mutual Authentication	×	×	✓

To provide a clear overview of the security properties of three representative lightweight authentication protocols for highly energy-constrained passive terminal devices, we present a qualitative comparison based on each protocol’s interaction logic and underlying mechanisms. Table 9 summarizes the resistance of each protocol to common attack vectors, including replay attacks, man-in-the-middle attacks, impersonation, key leakage, physical attacks, and energy-related threats. Note that this comparison is based on protocol design and mechanism analysis rather than empirical testing.

The qualitative analysis considers key mechanisms such as timestamps, nonces, PUF responses, HMAC codes, and physical layer detection, as well as energy management strategies. LMAP provides basic protection against replay and key attacks; LPUFA adds resistance to man-in-the-middle, physical, and eavesdropping threats through PUF-based authentication; and E²HA offers full mutual authentication along with robust defense against tampering and MITM. This comparison highlights the trade-offs among complexity, security coverage, and energy efficiency in ultra-low-power IoT deployments.

E Supplementary Evaluation Results

This section provides supplementary details of the proposed energy-driven task scheduling mechanism, including implementation examples, data analysis, and experimental results. The additional figures and explanations are intended to complement the main body of the paper by clarifying design choices, demonstrating performance trade-offs, and validating the effectiveness of the approach under dynamic energy conditions.

E.1 Task Performance Evaluation

This experiment systematically and comprehensively evaluates the temporal energy variation patterns, task execution characteristics, the detailed energy consumption of different scheduling strategies, and the significant influence of DVFS on energy savings along the local execution path of BFNs. On the experimental platform, the system’s residual energy exhibits significant fluctuations over time: initially, energy is relatively sufficient; during periods of intensive task execution, energy rapidly decreases, followed by oscillations around a low energy level. These observed energy fluctuation trends realistically reflect the highly dynamic energy consumption occurring during task execution, as well as transient energy recovery that may take place during intermittent low-power task phases in battery-free environments, thereby effectively illustrating both the discontinuity and inherent dynamism of the energy supply. Figure 6 visually presents these detailed fluctuation patterns.

Task-level analysis reveals that task complexity, data dependencies, and computational intensity significantly affect energy consumption and latency characteristics. The task scheduling mechanism makes decisions based on different task attributes to minimize energy consumption. In particular, Figure 7 compares the total energy consumption with and without DVFS, showing that the proposed dynamic scheduling mechanism combined with DVFS significantly reduces the power consumption of high-energy tasks. Experimental results demonstrate that the proposed energy-aware dynamic scheduling mechanism effectively schedules tasks, improving overall system energy efficiency and task execution performance, thus demonstrating its practical value in resource-constrained battery-free environments.

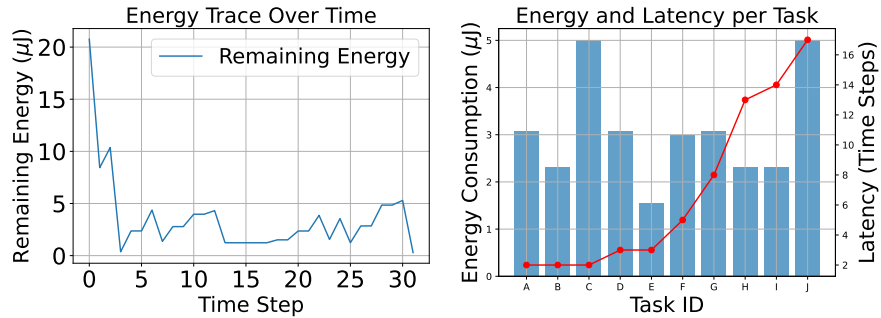


Figure 6: (Left) Residual energy fluctuation over Time in the BFNs; (Right) Detailed view of energy oscillations at low energy levels.

Summary

Together, these results further confirm the practicality and adaptability of the proposed mechanism in energy-constrained IoT environments.

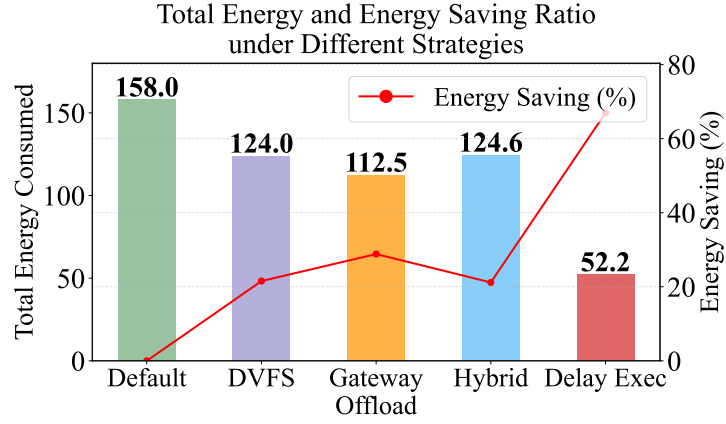


Figure 7: Total energy consumption with and without DVFS.

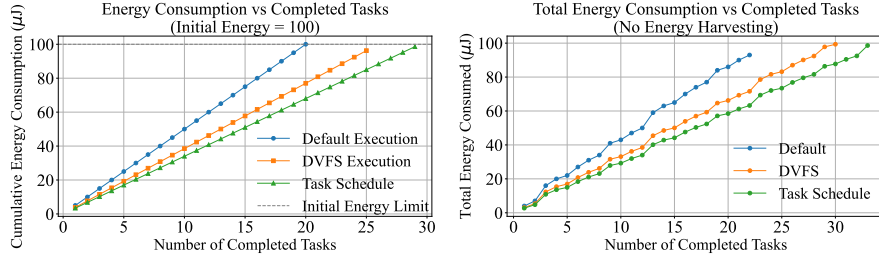


Figure 8: (Left) Comparison of cumulative energy consumption for different scheduling strategies without energy harvesting; (Right) Energy consumption improvement under energy harvesting conditions.